

How do transformers learn Hidden Markov Models?

A case study on the cylinder graph HMM.

March 5, 2026

1 Introduction

Next-token prediction—the task of estimating $P(X_{t+1} \mid X_{\leq t})$ —is the training objective underlying modern large language models (LLMs). Trained on this objective alone, LLMs develop rich internal representations—grammar, factual knowledge, reasoning patterns—none of which were explicitly programmed. A natural scientific question is: *what computational structure does the model build internally in order to make good predictions?*

To understand what transformers learn, a line of recent work has turned to synthetic datasets for which ground truth is known analytically. In this report we consider Hidden Markov Models (HMMs) as the data-generating process. An HMM is a two-layer stochastic process. A hidden (unobserved) variable Z_t evolves as a Markov chain on a finite state space of size N , with transition matrix $A_{ij} = P(Z_{t+1}=j \mid Z_t=i)$. At each time step, the current hidden state emits an observable token X_t from a vocabulary of size K , with probabilities $B_i(\mu) = P(X_t=\mu \mid Z_t=i)$. The observer sees only the token sequence $X_1, X_2, \dots$, but the hidden states remain hidden. Although the hidden process is Markovian, the observed token sequence is *not*: the marginal process $\{X_t\}$ exhibits autocorrelations that encode the latent dynamics.

A **transformer** takes as input a sequence of discrete tokens and outputs, at each position, a conditional probability distribution over the next token—it is an autoregressive model. The first layer is an *embedding*: each token is mapped to a continuous vector in $\mathbb{R}^d$. This embedding serves as a learned compression of the discrete vocabulary into a geometry where semantically related tokens lie nearby—a phenomenon famously illustrated by the discovery of meaningful directions in word-embedding spaces (e.g. “king” – “man” + “woman” $\approx$ “queen”). The embedding vectors are then processed by a stack of layers (self-attention and optional feedforward sublayers) that mix information across positions, and a final linear projection followed by a softmax produces the predicted next-token distribution $\hat{P}(X_{t+1} \mid X_{\leq t})$. A transformer with *context window* of length T can condition on at most T preceding tokens.

An HMM is a particularly natural test case because the forward algorithm allows us to compute $P(X_{1:T} \mid \lambda)$ exactly for any observation sequence. Moreover, the algorithm yields the *belief state*—the posterior distribution over hidden states given the observations so far—which is a sufficient statistic for next-token prediction and can be viewed as the analogue of an “internal world model” in the context of LLMs. The entropy rate of the process sets an absolute lower bound on the prediction loss, and the Bayes-optimal loss at finite context provides a tighter bound for models that can only look back a fixed number of steps.

We seek to answer the following questions about how transformers learn to model HMMs:

1. **How close can a transformer get to the information-theoretic optimum?** By comparing the trained model’s loss against the entropy rate and the Bayes-optimal loss, we can

cleanly decompose the gap into a *finite-context cost* (inherent to the architecture’s limited memory) and a *capacity gap* (how far the model’s learned strategy is from optimal given its context window).

2. **What internal representations does the model learn?** If the forward algorithm is the optimal computation, does the transformer discover something analogous to belief states? We can inspect the model’s learned embeddings and intermediate activations to look for signatures of the latent HMM structure.

We study these questions on a specific HMM called the “cylinder graph HMM” (Section 3). The key findings are:

- The best transformer (206k parameters) reaches within ~ 0.004 nats of the Bayes-optimal predictor at its context length, meaning it has essentially learned to perform near-optimal Bayesian filtering.
- Transformers consistently outperform n -gram models, which memorise token co-occurrence statistics, suggesting they learn something qualitatively beyond counting.
- The learned token embeddings spontaneously discover the depth-level structure of the HMM—tokens from the same hidden-state cluster are mapped to similar directions in embedding space—without any supervision about this structure.

The remainder of this report is organised as follows. Section 2 introduces the mathematical background on HMMs, belief states, and transformers. Section 3 defines the cylinder graph HMM. Section 4 derives the information-theoretic baselines. Section 5 describes the transformer architectures and training procedure. Section 6 presents results on scaling, architecture, and embedding structure. Section 7 concludes with open questions.

2 Technical Preliminaries

2.1 Hidden Markov Models

A Hidden Markov Model [1, 4, 2] is a doubly stochastic process consisting of an underlying Markov chain whose states are not directly observable, coupled with state-dependent observation distributions that produce the data we observe. Formally:

- **State space.** The hidden state Z_t takes values in a finite set S with $|S| = N$.
- **Observations.** The observable token X_t takes values in a finite set Ω with $|\Omega| = K$.
- **Markov assumption.** $P(Z_t \mid Z_{<t}) = P(Z_t \mid Z_{t-1})$.
- **Observation independence.** $P(X_t \mid Z_t, Z_{<t}) = P(X_t \mid Z_t)$.

The model is parameterised by the triple $\lambda = (A, B, \pi)$:

$$A_{ij} = P(Z_{t+1}=j \mid Z_t=i), \quad (\text{transition matrix}) \quad (1)$$

$$B_i(\mu) = P(X_t=\mu \mid Z_t=i), \quad (\text{emission probabilities}) \quad (2)$$

$$\pi_i = P(Z_1=i). \quad (\text{initial distribution}) \quad (3)$$

Note that while the hidden states are Markovian, the marginal process over observations is *not*.

As a concrete example, consider the Z1R process [2]: a three-state HMM with two observable tokens. State A deterministically emits “0” and transitions to B ; state B deterministically emits “1” and transitions to C ; state C emits a fair coin flip (“0” or “1”) and always returns to A . The observed sequence therefore has the pattern 0, 1, (random), 0, 1, (random), $\dots$ — a period-3 structure masked by the coin flip. Although each state transitions deterministically, an observer who sees only the tokens cannot distinguish whether a “0” came from state A or from the coin flip in state C . This simple example already illustrates the core challenge: optimal prediction requires tracking a *distribution* over hidden states, not just the most recent tokens.

The joint probability of a state sequence $Z_{1:T}$ and observation sequence $X_{1:T}$ factorises as

$$P(X_{1:T}, Z_{1:T} \mid \lambda) = \pi_{Z_1} B_{Z_1}(X_1) \prod_{t=2}^T A_{Z_{t-1}, Z_t} B_{Z_t}(X_t). \quad (4)$$

Rabiner [2] organised HMM computation into three canonical problems:

Problem	Given	Compute	Algorithm
Evaluation	$\lambda, X_{1:T}$	$P(X_{1:T} \mid \lambda)$	Forward algorithm
Decoding	$\lambda, X_{1:T}$	$\arg \max_{Z_{1:T}} P(Z_{1:T} \mid X_{1:T}, \lambda)$	Viterbi algorithm
Learning	$X_{1:T}$	$\arg \max_{\lambda} P(X_{1:T} \mid \lambda)$	Baum–Welch (EM)

In this paper we are concerned exclusively with the *evaluation* problem: the HMM parameters are known by construction, so neither Viterbi decoding nor Baum–Welch re-estimation is needed.

2.2 The Forward Algorithm

The evaluation problem asks: given λ and $X_{1:T}$, what is $P(X_{1:T} \mid \lambda)$? A naïve marginalisation over all hidden state sequences requires summing N^T terms. The key observation is that the joint probability (4) has a sequential structure: the contribution of each time step depends only on the previous hidden state. This allows us to accumulate the sum inductively—processing one time step at a time and carrying forward an N -dimensional vector that summarises all paths so far. This is the forward algorithm, which reduces the cost from $O(N^T)$ to $O(N^2T)$.

Define the *forward variable* $\alpha_t(j) := P(X_{1:t}, Z_t=j \mid \lambda)$ —the joint probability of observing $X_{1:t}$ and being in state j at time t .

Initialisation ($t = 1$):

$$\alpha_1(j) = \pi_j B_j(X_1), \quad j = 1, \dots, N. \quad (5)$$

Recursion ($t = 1, \dots, T-1$):

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^N \alpha_t(i) A_{ij} \right] B_j(X_{t+1}). \quad (6)$$

Termination:

$$P(X_{1:T} \mid \lambda) = \sum_{j=1}^N \alpha_T(j). \quad (7)$$

In matrix–vector form, writing α_t as a row vector and $D_t = \text{diag}(B_1(X_t), \dots, B_N(X_t))$:

$$\alpha_T = \pi^\top D_1 A D_2 A D_3 \cdots A D_T. \quad (8)$$

The complexity is $O(N^2T)$ in time and $O(N)$ in space when only the final likelihood is needed. For long sequences, $\alpha_t(j)$ underflows exponentially; standard remedies include log-space computation (log-sum-exp trick) or Rabiner’s scaling coefficients.

2.3 Belief States and Optimal Prediction

Define the *belief state*

$$\eta_t(j) := P(Z_t=j \mid X_{1:t}) = \frac{\alpha_t(j)}{\sum_{l=1}^N \alpha_t(l)}, \quad (9)$$

the posterior distribution over hidden states given the observations so far. Note the structural parallel with the transfer-matrix formalism in statistical mechanics: the forward variable α_t is propagated by a product of observation-dependent matrices $A D_t$ (Eq. 8), and the belief state is its normalised version, much as the dominant eigenvector of a transfer matrix encodes equilibrium probabilities.

The belief state obeys a recursive update. Define the *symbol-labelled transition matrix* $T_{ij}^{(x)} = A_{ij} B_j(x)$, which combines transition and emission into a single matrix for each observable symbol x . Then

$$\eta_{t+1} = \frac{\eta_t T^{(X_{t+1})}}{\|\eta_t T^{(X_{t+1})}\|_1}, \quad (10)$$

where $\|\cdot\|_1$ denotes the L^1 norm (sum of entries), ensuring η_{t+1} remains a valid probability distribution. The denominator is precisely the predictive probability

$$P(X_{t+1}=x \mid X_{1:t}) = \|\eta_t T^{(x)}\|_1 = \sum_{i,j} \eta_t(i) A_{ij} B_j(x). \quad (11)$$

Sufficiency. The next-token distribution $P(X_{t+1} \mid X_{1:t})$ depends on $X_{1:t}$ *only* through η_t . The belief state is therefore a *sufficient statistic* for next-token prediction: any system that predicts optimally must implicitly compute a representation equivalent to the belief state.

2.4 Belief-State Geometry

The belief state η_t lives on the probability simplex Δ^{N-1} . Each new observation x updates the belief via the map $f^{(x)}(\eta) = \eta T^{(x)} / \|\eta T^{(x)}\|_1$. The collection of maps $\{f^{(x)}\}_{x \in \Omega}$ —one per observable token—forms an Iterated Function System (IFS) on the simplex. For HMMs where a given state can emit the same symbol while transitioning to multiple distinct states (so-called *nonunifilar* HMMs), the set of reachable belief states generically has a fractal structure; see Shai et al. [5] for a detailed treatment in the context of transformers. We return to this geometry when examining the transformer’s learned embeddings (Section 6) and in the open questions (Section 7).

2.5 Transformers

A transformer [3] is a sequence-to-sequence neural network built from three main components. First, a *token embedding* layer maps each discrete token to a continuous vector in $\mathbb{R}^d$; a *positional encoding* is added so the model can distinguish positions. Second, a stack of *self-attention layers* computes, at each position, a data-dependent weighted average of representations from all preceding

positions (enforced by a *causal mask* that prevents attending to the future). Each attention layer uses multiple *heads* that attend to different aspects of the input in parallel. Attention layers may be interleaved with pointwise *feedforward* (MLP) sublayers that introduce additional nonlinearity. Finally, a linear projection followed by a softmax produces the predicted next-token distribution.

The model is trained *autoregressively*: for each position t in a training sequence, the cross-entropy between the model’s prediction $\hat{P}(X_t \mid X_{<t})$ and the true next token is minimised. A transformer with context window of length T conditions on at most T preceding tokens. The key challenge for our setting is that the transformer must learn to approximate the belief-state computation of Section 2.3 using only the observed tokens—it has no direct access to the hidden states.

3 The Cylinder Graph HMM

The cylinder graph HMM is designed to test whether transformers can discover structured latent representations from data alone. The key idea is to build an HMM whose hidden states have a notion of “semantic similarity”—states within the same depth level emit overlapping token distributions (analogous to semantically related words appearing in similar contexts in natural language)—while states at different depth levels emit from disjoint clusters. If the transformer’s embedding layer discovers this structure, tokens from the same depth cluster should have high cosine similarity in embedding space, even though no such grouping is provided during training.

Concretely, the model is an instance of the general HMM framework in Section 2.1, with $N = 18$ hidden states and $K = 48$ observable tokens. The hidden-state graph has the topology of a discrete cylinder with *width* $n = 6$ (nodes per level) and *depth* $D = 3$ (levels), giving $n \times D = 18$ states in total.

3.1 Transition structure

Each level $l \in \{0, 1, 2\}$ contains $n = 6$ nodes arranged in a ring. From a node (l, j) at depth l and angular position j the chain can jump to three neighbours:

- **Nearest neighbour (NN):** $(l, j+1 \bmod n)$ with weight w , where $w \sim \text{Uniform}(0, 1-p)$;
- **Next-nearest neighbour (NNN):** $(l, j+2 \bmod n)$ with weight $1 - p - w$;
- **Next layer:** $((l+1) \bmod D, j)$ with fixed probability $p = 0.1$.

The intra-layer weights w are drawn independently for each node from $\text{Uniform}(0, 1-p)$ once at initialisation (seed 42) and held fixed thereafter. The inter-layer probability p controls how often the chain cycles through the depth levels. Because transitions are predominantly intra-layer, consecutive tokens are likely to come from the same depth cluster—giving the observed sequence a form of local “semantic coherence” that the transformer can exploit.

3.2 Emission structure

The vocabulary has $|V| = D \times K_{\text{cl}} = 3 \times 16 = 48$ tokens, partitioned into three clusters of 16 tokens each (one cluster per depth level). Each hidden state at depth l emits tokens only from cluster l , with emission probabilities drawn from a symmetric Dirichlet distribution with concentration parameter $\alpha = 0.3$. The small α makes the per-state distributions sparse, so different states at the same depth emit noticeably different token profiles. Within a depth cluster, tokens emitted by

nearby hidden states (on the ring) will tend to co-occur in short windows, injecting fine-grained similarity structure that goes beyond the coarse depth-level grouping.

4 Information-Theoretic Baselines

4.1 Entropy rate

The *entropy rate* of a stationary stochastic process $\{X_t\}$ is

$$h = \lim_{T \rightarrow \infty} \frac{1}{T} H(X_1, \dots, X_T) = \lim_{T \rightarrow \infty} H(X_T \mid X_1, \dots, X_{T-1}). \quad (12)$$

For the second equality, we have used the chain rule of entropy. The entropy rate is the irreducible uncertainty per token for a predictor with access to the infinite past, and therefore serves as the information-theoretic lower bound on the cross-entropy loss achievable by any model.

For an HMM with transition matrix A and emission matrix B , the entropy rate can be estimated empirically by generating a long sequence $(x_1, \dots, x_M)$, running the forward algorithm (Section 2.2) to compute the predictive probabilities $P(x_t \mid x_1, \dots, x_{t-1})$ via Eq. (11), and averaging:

$$\hat{h} = -\frac{1}{M - M_{\text{burn}}} \sum_{t=M_{\text{burn}}+1}^M \log P(x_t \mid x_1, \dots, x_{t-1}), \quad (13)$$

where the first M_{burn} tokens are discarded to allow the belief state to converge from an arbitrary prior. For the cylinder graph HMM with the parameters specified above, this yields $\hat{h} \approx 2.44$ nats/token.

4.2 Bayes-optimal loss at finite context

The entropy rate is the minimum achievable loss for a predictor with access to the *infinite* past. A transformer with context window T , however, can only condition on the most recent T tokens. To disentangle the loss due to *finite context* from the loss due to *limited model capacity*, we compute the Bayes-optimal next-token loss at finite context: the best any predictor can do when restricted to a window of length k .

Given a context window $(x_1, \dots, x_k)$, the Bayes-optimal prediction for the next token is

$$P(X_{k+1}=\mu \mid x_1, \dots, x_k) = \sum_{i,j} \eta_k(i) A_{ij} B_j(\mu), \quad (14)$$

where η_k is the belief state over hidden states after processing the context, computed via the recursive update in Eq. (10). The belief state is initialised from the stationary prior $\eta_0(i) = \pi(i)$. Each observed token updates the belief via

$$\tilde{\eta}_t(j) = \sum_i \eta_{t-1}(i) T_{ij}^{(x_t)}, \quad \eta_t(j) = \frac{\tilde{\eta}_t(j)}{\sum_{j'} \tilde{\eta}_t(j')}. \quad (15)$$

The normalising constant $Z_t = \sum_{j'} \tilde{\eta}_t(j')$ equals the predictive probability $P(x_t \mid x_1, \dots, x_{t-1})$, so the conditional log-likelihood of the context decomposes as $\log P(x_1, \dots, x_k) = \sum_{t=1}^k \log Z_t$.

After k update steps the belief η_k encodes all information that the context provides about the current hidden state. Plugging it into Eq. (14) yields the Bayes-optimal predictive distribution and hence the minimum achievable loss at context length k :

$$L^*(k) = \mathbb{E}[-\log P(X_{k+1} | X_1, \dots, X_k)], \quad (16)$$

where the expectation is over sequences drawn from the HMM.

For a *fixed* context window, each length- k chunk is treated independently: the belief is reset to π at the start of every window. This is exactly the information available to a transformer with context length k . For the *infinite-context* baseline, the forward algorithm runs over the entire sequence without resetting, so the belief accumulates information from arbitrarily far in the past. The difference

$$\Delta(k) = L_{\text{fixed}}^*(k) - L_{\infty}^* \quad (17)$$

isolates the cost of the finite context window. As k grows beyond the mixing time of the chain, $\Delta(k) \rightarrow 0$.

The cross-entropy loss of a trained transformer $\hat{L}$ can therefore be decomposed as

$$\underbrace{\hat{L}}_{\text{transformer loss}} = \underbrace{h}_{\text{entropy rate}} + \underbrace{\Delta(k)}_{\text{finite-context cost}} + \underbrace{D_{\text{KL}}}_{\text{model capacity gap}}, \quad (18)$$

where $D_{\text{KL}} = \hat{L} - L_{\text{fixed}}^*(k) \geq 0$ measures the sub-optimality of the transformer relative to the Bayes-optimal predictor at the same context length.

4.3 N -gram baseline

An n -gram model estimates the next-token distribution by conditioning on a fixed-length context of $n-1$ preceding tokens. The maximum likelihood estimate is

$$\hat{P}_{\text{MLE}}(w | w_{t-n+1}, \dots, w_{t-1}) = \frac{C(w_{t-n+1}, \dots, w_{t-1}, w)}{C(w_{t-n+1}, \dots, w_{t-1})}, \quad (19)$$

where $C(\cdot)$ denotes the count of the n -gram (or $(n-1)$ -gram context) in the training corpus. When a context has never been observed, we back off to shorter contexts until a match is found (see Appendix A for backoff details). The implementation is in `src/ngram_baseline.py`.

4.4 Summary of baselines

Table 1 summarises the theoretical and empirical baselines.

5 Transformer Architectures

We train causal transformer language models (implemented via `TransformerLens HookedTransformer`) to predict the next token in sequences of length $T = 16$ generated by the cylinder HMM.

The hyperparameter sweep covers:

Hyperparameter	Values
Number of layers L	2, 4
Embedding dimension d	4, 8, 16, 32, 48, 64
Number of heads H	1, 2, 4
Architecture	attention-only, full (attention + MLP)
Normalisation	none, LayerNorm

Baseline	Loss (nats)	Description
Entropy rate h	≈ 2.44	Infinite-context lower bound
Bayes-optimal $L^*(k=16)$	≈ 2.47	Best fixed-context ($k=16$) predictor
Best n -gram (trigram)	2.57	Best count-based model (test CE)
Bigram	2.67	Single-token context baseline
Unigram	3.65	No context (marginal frequencies)

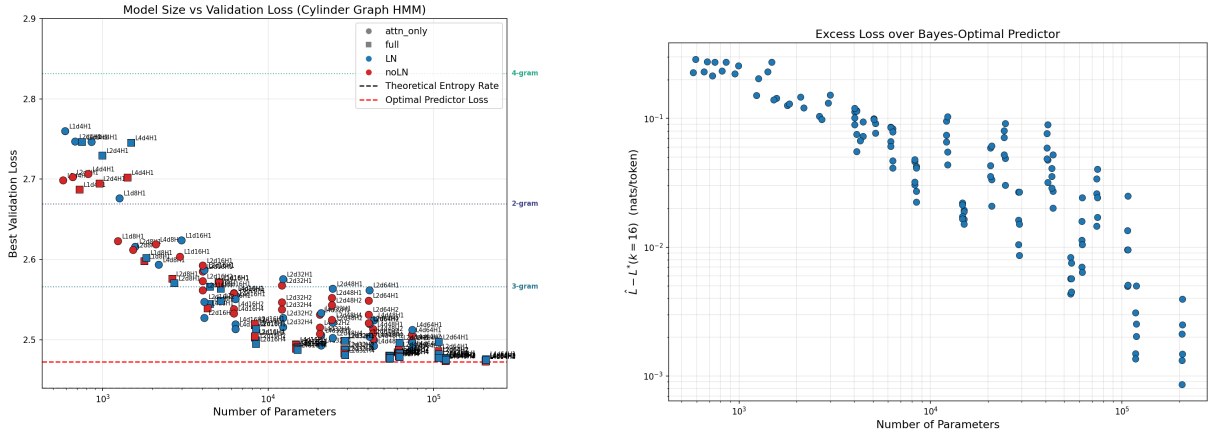
Table 1: Information-theoretic and count-based baselines for the cylinder graph HMM. The finite-context cost $\Delta(16) \approx 0.03$ nats is small, indicating that the context window $k = 16$ is already sufficient for near-optimal inference.

All models use ReLU activations, tied input/output embeddings, and $d_{\text{mlp}} = 4d$ for full architectures. Training uses AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay 0.01, learning rate 10^{-3}) with batch size 128 for 10 epochs. A total of 135 models were trained on a dataset of ~ 618 k sequences (10^7 tokens).

6 Results

6.1 Model size vs. validation loss

Figure 1 plots the best validation loss achieved by each model against its parameter count.



(a) Validation loss vs. parameter count. Circles: attention-only; squares: full (attention + MLP). Blue: LayerNorm; red: no normalisation. Dashed lines mark the entropy rate $h \approx 2.44$ nats/token, the Bayes-optimal loss $L^*(k=16) \approx 2.47$ nats, and the best n -gram baseline (trigram, 2.57 nats).

(b) Excess loss $\hat{L} - L^*(k=16)$ on a log-log scale. All architectural distinctions are suppressed to highlight the overall scaling trend with model size.

Figure 1: Model size versus validation loss for 135 transformer configurations on the cylinder graph HMM. (a) Absolute validation loss. (b) Excess loss over the Bayes-optimal predictor at context length $k = 16$.

Validation loss decreases roughly log-linearly with parameter count up to $\sim 10^5$ parameters,

after which gains diminish. At every scale, full transformers (with MLP layers) outperform their attention-only counterparts, and all top-10 models use $L = 4$ layers. LayerNorm has a negligible effect compared to architecture and scale. The best model (L4, $d=64$, $H=4$, full, no-LN; $\sim 206k$ parameters) reaches ≈ 2.473 nats/token. By the decomposition in Eq. (18), most of the 0.034 nat gap above the entropy rate is attributable to the finite context window ($\Delta(16) \approx 0.03$ nats), leaving only ~ 0.004 nats of model capacity gap.

6.2 N -gram comparison

Table 2 reports the training and test cross-entropy loss for n -gram orders 1 through 6 (without regularisation; see Appendix A for smoothed results).

n	Train CE	Test CE	# Contexts
1	3.654	3.652	1
2	2.667	2.669	48
3	2.551	2.566	1 540
4	2.470	2.832	39 531
5	2.236	4.781	443 415
6	1.713	8.618	1 953 864

Table 2: N -gram cross-entropy (nats/token) without regularisation. For $n \geq 4$, test loss diverges due to overfitting. The best test performance is the trigram ($n = 3$).

The largest drop in cross-entropy occurs from the unigram to the bigram ($3.65 \rightarrow 2.67$ nats), reflecting the strong first-order correlations induced by the HMM’s clustered emission structure: observing a single token already reveals the current depth level, narrowing the effective vocabulary from 48 to 16. The trigram provides a modest further gain, approaching the Bayes-optimal loss. The best n -gram model (trigram, test CE = 2.57) is outperformed by transformers with embedding dimension $d \geq 16$, suggesting that these models learn to approximate belief-state inference beyond simple token co-occurrence statistics. Conversely, very small transformers ($d = 4$) fail to beat the trigram, indicating that a minimum model capacity is needed before the transformer can surpass what a count table captures directly.

6.3 Embedding structure

Figure 2 shows the cosine-similarity matrix and ℓ_2 norms of the learned token-embedding vectors from a representative model. The 3×3 block structure in the similarity matrix directly mirrors the depth-level clustering of the vocabulary: tokens within the same depth cluster are mapped to similar embedding directions, while cross-cluster pairs are near-orthogonal or anti-correlated. This confirms that the model discovers the latent depth structure without supervision.

Cluster similarity across architectures To quantify how consistently the embedding geometry reflects the depth-level structure, we compute the cosine similarity between cluster centroids across all 135 trained models. For each model we extract the embedding matrix $W_E \in \mathbb{R}^{48 \times d}$ and compute three centroids

$$\mathbf{c}_l = \frac{1}{16} \sum_{v=16l}^{16l+15} W_E[v, :], \quad l \in \{0, 1, 2\}, \quad (20)$$

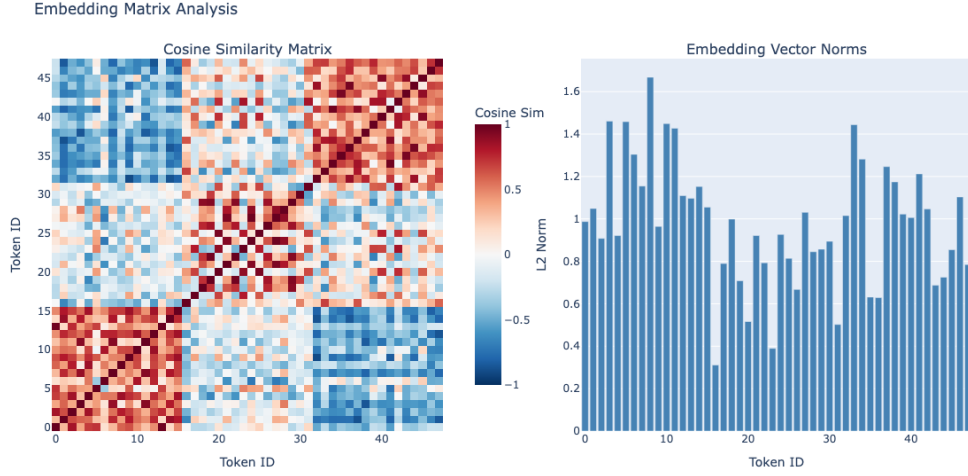


Figure 2: Left: cosine similarity matrix of the $48 \times d$ embedding matrix. The visible 3×3 block structure corresponds to the three depth levels. Right: ℓ_2 norm of each token’s embedding vector.

followed by the three pairwise cosine similarities $\text{sim}(\mathbf{c}_0, \mathbf{c}_1)$, $\text{sim}(\mathbf{c}_0, \mathbf{c}_2)$, and $\text{sim}(\mathbf{c}_1, \mathbf{c}_2)$.

Figure 3 shows these similarities as a function of validation loss. All three pairs are predominantly negative (mean ≈ -0.4), confirming that the learned embeddings separate the three depth clusters into roughly opposing directions in embedding space. Models that achieve low validation loss (left side of the plot) cluster tightly in the range -0.35 to -0.50 , while poorly performing models exhibit much higher variance—some cluster pairs even become positively correlated. This suggests that learning to separate the depth clusters in embedding space is necessary for good predictive performance, but the degree of separation saturates once the model is sufficiently expressive.

7 Summary and Open Questions

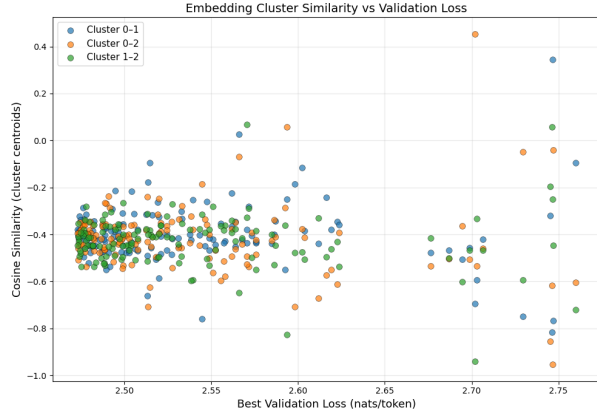
The cylinder graph HMM provides a controlled testbed for studying how transformers learn to perform approximate Bayesian filtering over a structured latent space. Larger, deeper, full-architecture models approach but do not reach the entropy rate, suggesting fundamental limits tied to the autoregressive architecture and finite context.

Open directions include:

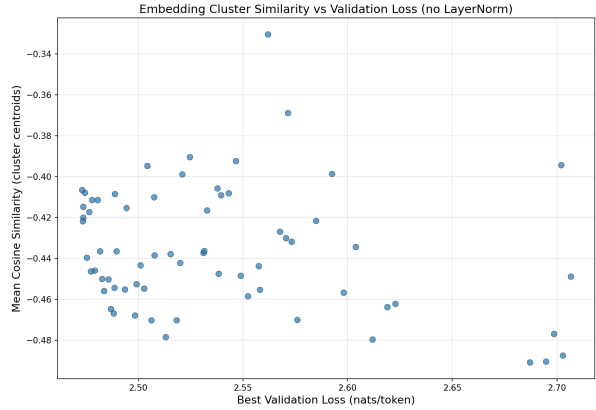
- Characterising the belief-state geometry learned by the residual stream (cf. the fractal attractor structure discussed in Section 2.4 and [5]).
- Ablation studies on context length to quantify the loss attributable to finite T .
- Mechanistic interpretability of attention heads—do specific heads specialise in tracking depth transitions versus intra-layer dynamics?

A N -gram Backoff Details

Backoff When an $(n-1)$ -gram context has never been observed in training, the smoothed estimate degenerates to $1/|V|$. To avoid this, we employ a simple backoff strategy: if the full $(n-1)$ -gram



(a) All three pairwise cosine similarities between cluster centroids, plotted for each of the 135 trained models. Poorly performing models show high variance, with some cluster pairs becoming positively correlated.



(b) Mean of the three pairwise similarities per model, restricted to architectures without LayerNorm (67 models). The trend is flat, with most models in $[-0.49, -0.35]$.

Figure 3: Cosine similarity between depth-cluster centroids of the embedding matrix versus best validation loss. (a) Per-pair similarities for all models. (b) Mean similarity for noLN models only.

context is absent from the count table, we fall back to the $(n-2)$ -gram model, then the $(n-3)$ -gram, and so on down to the unigram. The first context length that appears in the training counts is used with its smoothed estimate.

References

- [1] L. E. Baum and T. Petrie, “Statistical Inference for Probabilistic Functions of Finite State Markov Chains,” *Annals of Mathematical Statistics*, 37(6):1554–1563, 1966.
- [2] L. R. Rabiner, “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition,” *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” *Advances in Neural Information Processing Systems*, 30, 2017.
- [4] L. R. Rabiner and B.-H. Juang, “An Introduction to Hidden Markov Models,” *IEEE ASSP Magazine*, 3(1):4–16, 1986.
- [5] A. S. Shai, S. E. Marzen, L. Teixeira, A. Gietelink Oldenziel, and P. M. Riechers, “Transformers Represent Belief State Geometry in Their Residual Stream,” in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. [arXiv:2405.15943](#).